House Price Prediction using Ensemble Models

1. Introduction

The goal of this project is to predict house prices using the Kaggle *House Prices: Advanced Regression Techniques* dataset.
The dataset contains 79 explanatory variables describing structural, qualitative, and locational aspects of residential properties.

The objective is to build and compare multiple regression models, with emphasis on tree-based ensemble methods, and evaluate their performance using RMSLE.
Special attention is given to model generalization, computational cost, and interpretability

.

2. Data Exploration & Preprocessing

The dataset consists of:

- 1460 training samples
- 1459 test samples

The target variable (*SalePrice*) exhibits:

- Mean: $180,921
- Median: $163,000
- Standard Deviation: $79,442
- Skewness: 1.88

The strong positive skewness indicates that most houses are concentrated in lower price ranges, with fewer high-priced outliers.

This justifies the use of log transformation, as it reduces skewness and stabilizes variance.

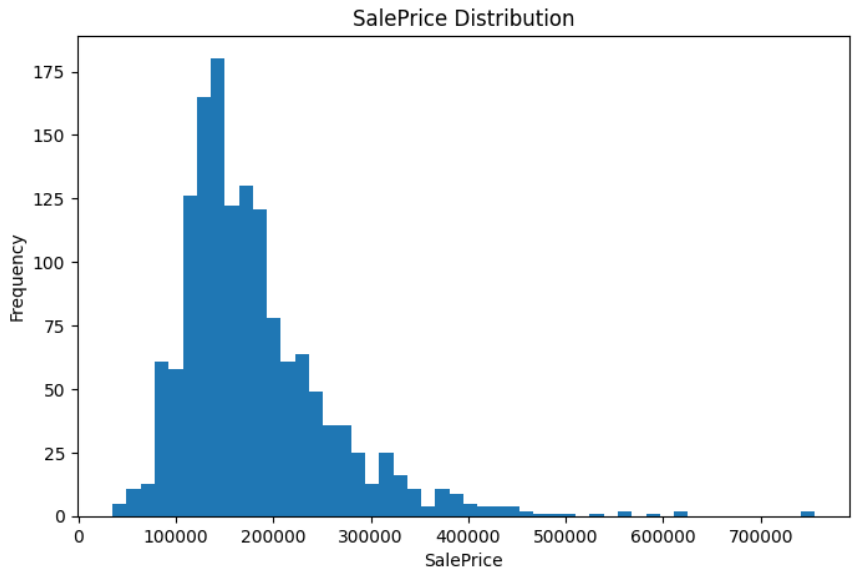
Missing values were handled systematically:

- Numerical features → median imputation (robust to outliers)
- Categorical features → "None", preserving semantic meaning

Missing values in features such as *PoolQC* or *Alley* represent the absence of the feature rather than missing data.

Categorical variables were encoded using One-Hot Encoding within a pipeline, ensuring consistency across all models

.



3. Metric Selection (RMSLE)

RMSLE was selected due to the nature of the target variable:

- House prices span a wide range ($35k – $755k)
- Absolute errors are less meaningful than relative errors

RMSLE focuses on **percentage error**, ensuring fair evaluation across different price ranges.

Additionally:

- Log transformation reduces heteroscedasticity
- Improves stability for tree-based models

Interpretation:

An RMSLE of 0.146 corresponds to approximately 14% relative error, meaning a $100,000 house would have an error of around ±$14,000.

4. Baseline Models

Two baseline models were evaluated:

- Dummy Regressor (mean prediction)
- Linear Regression

Results:

- Dummy: 0.3992
- Linear Regression: 0.1460

This represents a **63% improvement**, indicating strong predictive structure in the data.

Linear Regression captures significant linear relationships between features and price, demonstrating that the dataset contains meaningful signal rather than noise.

5. Decision Tree

As model complexity increases (depth):

- Training error continuously decreases
- Validation error decreases up to depth ≈ 6–7, then stabilizes

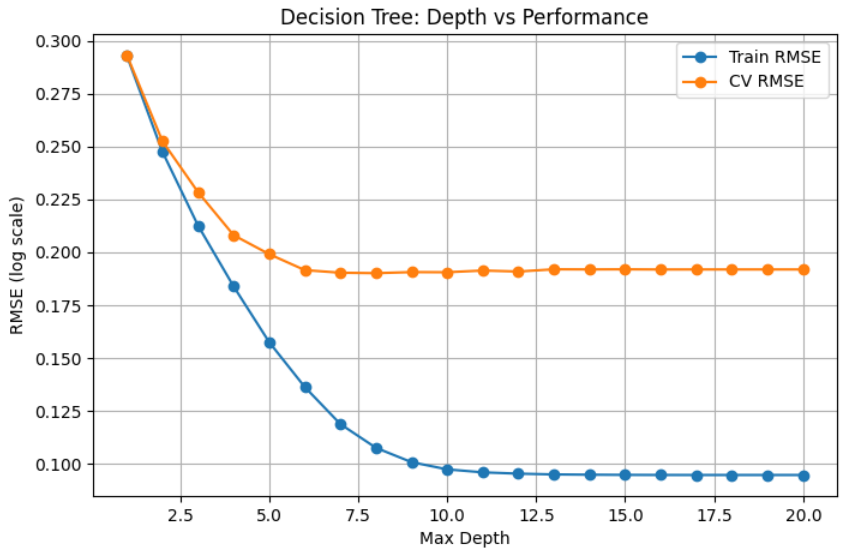
This indicates **overfitting**, where the model starts learning noise instead of general patterns.

The optimal depth (~6–7) balances bias and variance.

Final performance:

- RMSLE ≈ 0.194

The Decision Tree underperforms compared to Linear Regression, suggesting that a single tree lacks stability and generalization capability.



6. Random Forest

Random Forest reduces overfitting by averaging multiple trees.

Key observations:

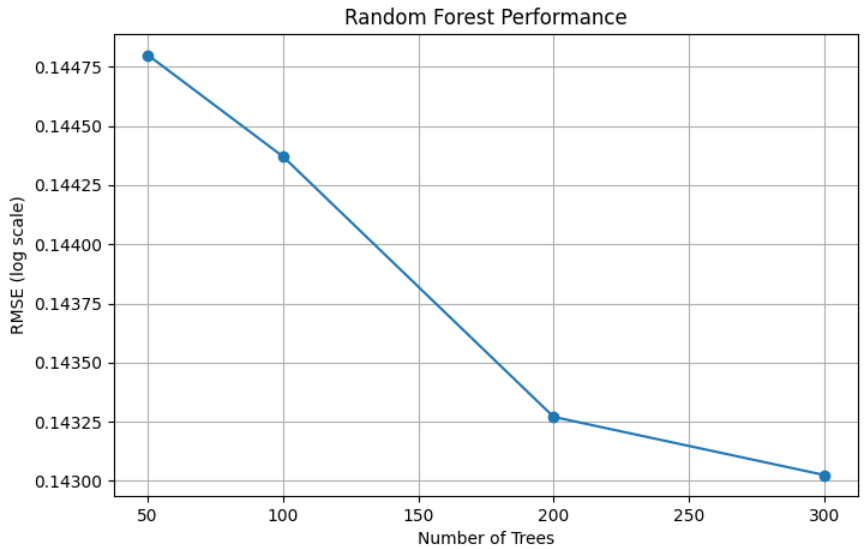
- Performance improves significantly compared to a single tree
- Gains diminish after ~200 trees

Increasing from 200 to 300 trees yields minimal improvement while increasing computational cost.

Final performance:

- RMSLE ≈ 0.143

This confirms that ensemble averaging effectively reduces variance but does not drastically improve accuracy beyond a certain point.



7. Gradient Boosting

Gradient Boosting achieved the best overall performance.

Best configuration:

- learning_rate = 0.10
- n_estimators = 300

Results:

- RMSLE ≈ 0.123
- Std ≈ 0.008

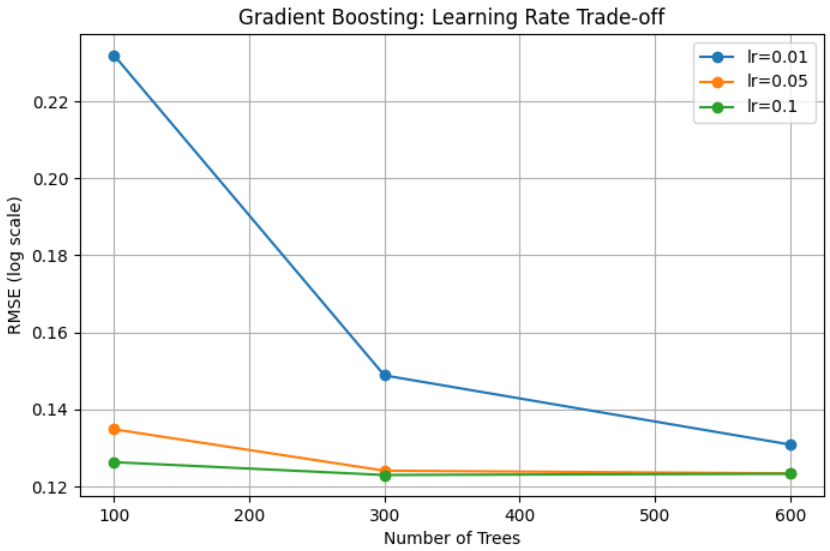
Key insights:

- Smaller learning rates require more trees to converge
- Larger learning rates converge faster but risk overshooting

The chosen configuration provides the best balance between convergence speed and generalization.

Additionally:

- Increasing trees beyond 300 leads to diminishing returns
- Training time increases without meaningful performance gain



8. XGBoost

XGBoost demonstrates strong performance and efficiency.

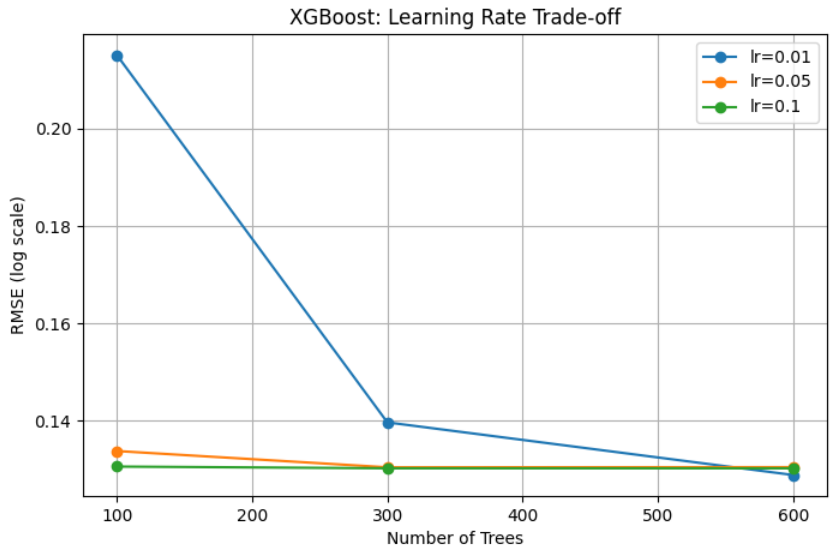
Key observations:

- Fast convergence, especially at higher learning rates
- Performance stabilizes around 0.129–0.130

Despite its efficiency, XGBoost did not outperform Gradient Boosting in this setup.

Possible reasons:

- Limited hyperparameter tuning
- Dataset characteristics favoring sklearn GB implementation



9. Feature Importance

Permutation importance reveals:

Top features:

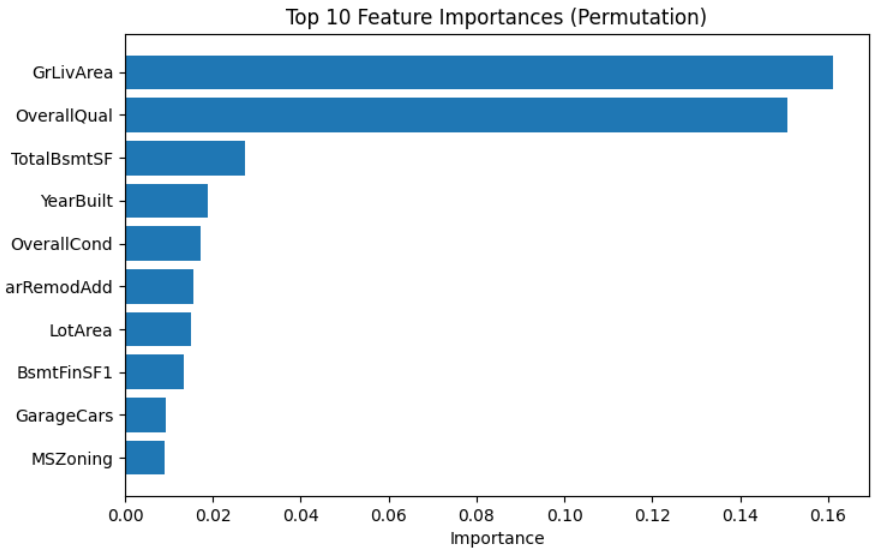
- GrLivArea (living area)
- OverallQual (construction quality)

These features dominate, indicating that structural characteristics are the primary drivers of price.

Secondary features:

- Basement size
- Year built
- Garage capacity

The model relies more on physical attributes than location-specific variables.



10. Residual Analysis

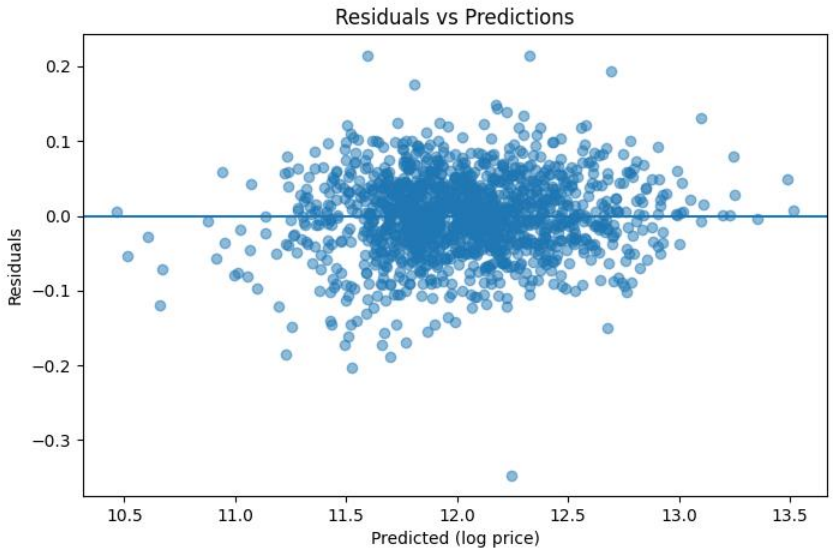
The residual plot shows:

- Mostly random distribution (good generalization)
- Slight increase in variance for higher values

This indicates mild heteroscedasticity, where prediction error increases for expensive houses.

Reason:

- Fewer high-priced samples
- More complex patterns in luxury properties



11. Model Comparison & Runtime

There is a clear trade-off:

- Simple models → fast but less accurate
- Ensemble models → slower but more accurate

Key observation:

Gradient Boosting provides the best accuracy, while XGBoost offers a strong balance between speed and performance.

12. Final Model & Submission

The best model (Gradient Boosting) was retrained on the full dataset.

Predictions were transformed back using:

$\text{SalePrice} = \exp(\hat{y}) - 1$

The final output was saved in Kaggle submission format.

---

13. Final Conclusion

Gradient Boosting achieved the best overall performance, confirming that boosting methods are highly effective for structured tabular data.

XGBoost offered competitive performance with lower computational cost, while Linear Regression proved to be a strong baseline.

Overall, ensemble methods significantly outperform simpler models, especially in datasets with nonlinear relationships and complex feature interactions.